

Image Multi-classification

Cooper DelGaudio

August 26, 2022

Abstract

We review supervised machine learning, neural networks, and convolutional layers and apply them to a multi-classification problem with three categories to create an algorithm to diagnose chest X-rays. We then discuss the product of the model. We find that our model achieves better results than our baseline.

Intro

Machine learning, an artificial intelligence that allows software applications to be able to predict outcomes without being explicitly coded to do so, can be split up into supervised learning, unsupervised learning, semi-supervised learning, and reinforcement learning [2]. This paper will focus on supervised learning, which is when the computer is trained on a labeled set of data.

The specific problem we are faced with is diagnosing chest X-rays to see whether someone has Covid-19, Pneumonia (we are assuming this Pneumonia is unrelated to Covid-19), or there are No findings. Covid-19, which was declared a pandemic by WHO in March of 2020, is a respiratory disease that can cause inflammation in the lungs [3]. Similarly, Pneumonia is an infection that causes inflammation in one's lungs [6]. Consider Figure 1; that table displays an example of an X-ray for each label.

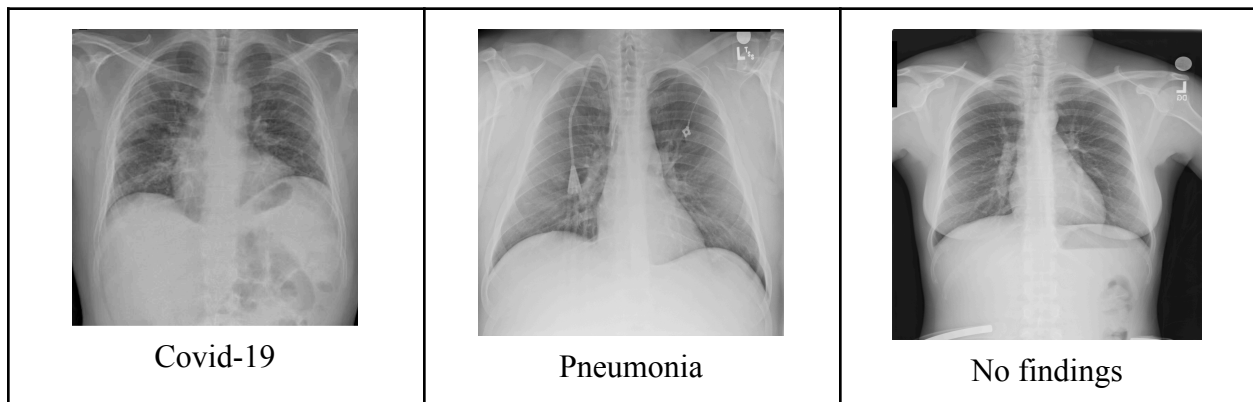


Figure 1: Example of data from data set

In supervised machine learning, we consider datasets, a collection of examples, to train our models. In this specific scenario, the examples are chest X-rays. Each example possesses features and labels, the labels being Covid-19, Pneumonia, and No findings [5]. To train our supervised

learning model, the data is split up into a training set and a validation set, which helps better our model. The goal of supervised learning is to be able to predict the correct label when input the features for a certain example.

This project belongs to a specific subset of machine learning, which is image recognition, as the program will be interpreting chest X-rays. Thus, when we give the program the specific image, we expect to get the image's corresponding label in a trained model.

In the first section of this paper, we will discuss related works and how they compare to our final model. Then we move into a breakdown of our data. The third section consists of the overview of the type of problem we faced. After this, we will talk about a breakdown of the parts of our model.

Related Works

There are not that many submissions of code for this specific dataset on Kaggle, explaining why this section will be relatively short. The program created by Pranjall Kumar uses over 20 layers in order to get a result of a loss of 0.0016, an accuracy of 1.0000, a val loss of 0.5810, and a val accuracy of 0.8632 [5]. To put in perspective to our model, we did not use nearly as many layers in our program, and our results were just a little less accurate. This demonstrates that our program prioritizes efficiency in terms of run time, which could as a result lose some accuracy.

Data set

This data set is titled "Covid X-rays" on Kaggle [5]. All the data is stored in the folder titled "X-rays," which contains three folders: test, train, and val. Each folder then contains three additional folders, one for each category. Once again, we are assuming the folders entitled Covid-19 and Pneumonia are completely unrelated. The training set came with 200 examples for Covid-19, 280 examples for Pneumonia, and 280 examples for No findings. Both the test and validation sets came with 25 examples of Covid-19, 35 examples of Pneumonia, and 35 examples of No findings. Due to the fact that the training set contained 760 total examples and the validation set only contained 95 examples, we decided to combine the validation and test sets to create a new validation set that contained 190 examples. We did this to maintain a balance of 80% training set and 20% validation set as we thought this would be a fair breakdown of the data. Evaluating our model using the test set is beyond the goal of our program, which is to dive into the basics of machine learning and create a program where we use just the accuracy from training the model to evaluate our model.

After organizing the data into the `X_train` and `y_train` arrays, we discover that the `X_train` array is a 3-D tensor and that the `y_train` array is a 1-D tensor. In the `y_train` array which carries the

labels, we fill it with 0's, 1's, and 2's. The 0's represent No findings, the 1's represent Covid-19, and the 2's represent Pneumonia. Table 1 displays the arrays belonging to a specific image in the 7th index.

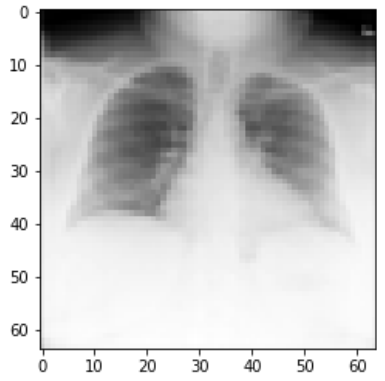
X_train[7]	y_train[7]	X-ray
<pre>array([[18., 4., 3., ..., 5., 5., 3.], [19., 4., 3., ..., 6., 7., 4.], [16., 3., 2., ..., 3., 5., 3.], ..., [222., 234., 237., ..., 237., 237., 232.], [208., 226., 232., ..., 235., 232., 227.], [199., 221., 227., ..., 236., 233., 225.]])</pre>	0	

Table 1: Example of the arrays behind an X-ray with No findings

Since there is no missing data, all tensors appear to be the same size, and all the data appears to be on the same scale of 0-255 (the data appears this way in Table 1 as the data values in the X_train array are between 0-255), no data cleanup was necessary prior to beginning the coding of the program.

This data set is also fairly balanced, which means that there are roughly the same amount of examples for each category. The 200 examples in the Covid-19 category is pretty close to the 280 examples in the Pneumonia and No findings categories in the training set. The same goes with the validation set as well, as the number of examples for each label is roughly the same.

Before we started to program the algorithm, we had to first scale the data. The scaled lists of data consist of numbers that are not spread out too far apart, nor clumped together. The mean of the scaled list is 0 and the standard deviation is 1.

Multi-classification problem

A multi-classification problem is defined as a categorical problem with more than two possible categories. For this particular problem, the categories are No findings, Covid-19, and Pneumonia.

Categorical-cross-entropy error

While we will not go into great detail about the categorical-cross-entropy error in this paper, we will briefly discuss what it is.

Let k be the number of categories. In this model, $k = 3$. Let $y_1, y_2, \dots, y_k$ be the labels (after one-hot encoding) such that one of them is equal to 1 and the rest are equal to 0. Let $\hat{y}_1, \hat{y}_2, \dots, \hat{y}_k$, be the predicted values of our model such that they are positive numbers and they add up to 1.

The categorical-cross-entropy error for our model is

$$\text{CCE} = - \sum_{i=1}^k (y_i \log(\hat{y}_i))$$

The closer $\hat{y}_1, \hat{y}_2, \dots, \hat{y}_k$ are to $y_1, y_2, \dots, y_k$, the smaller the error will be. With a smaller error, the prediction is better.

The mean categorical-cross-entropy error is the average of the categorical-cross-entropy errors on the examples in the data set.

Convolutional Neural Network

In order to create the convolutional neural network in our project, we had to import Sequential, Dense, Conv2D, MaxPool2D, and Flatten from Keras to create the layers to train the program.

Neural Networks work for both classification and non-classification problems, and the problem we are trying to solve is a multi-classification problem. Neural Networks are functions that predict the labels in a set of data, in which they take as input the scaled features of an example and produce the predicted label(s) as output. There are many different types of Neural Networks, but for our project, we use a Convolutional Neural Network. A Convolutional Neural Network takes an image as input and assigns importance to different aspects in the image to help differentiate one image from another [9].

There are many layers that went into creating the Convolutional Neural Network we created, and we will briefly describe each layer.

1. Sequential layer: takes the tensor as input—for our project, that would be a 3-D tensor.
2. Conv2D layer: This layer has filters; a filter is a concentration of kernels (kernels extract certain features from an image) [4]. Common kernels are 1x1, 3x3, 5x5, and 7x7 [7]. The stride for this layer is (1,1), which means that the filters are applied, then it moves one pixel to the right, then the filters are applied again [7]. This process is repeated until we reach the far-right border, then it goes down one pixel as the stride is just (1,1). This layer also has “same padding,” which means that the size of the output tensor is the same size as the input tensor. Lastly, we used relu as the activation function.
3. MaxPool2D layer: The pooling layer we used has a size of (2,2) which means that the program checks a 2x2 matrix for the largest number and puts that number in the output tensor for this layer. The default stride for this layer is the same as the pool size [10]. Like the Conv2D layer, this takes in a 3-D tensor as input and outputs a 3-D tensor.
4. Flatten layer: This layer acts as the transition between the convolutional layers and the fully connected layers. This layer flattens the 3-D tensors into 1-D tensors. If the input tensor was called A, this layer essentially does `A.reshape(-1)`.
5. Dense layer: This layer has a number of nodes that varies depending on the model. If the dense layer is used as the output layer, the number of nodes matches the number of categories and uses softmax as its activation function. If this layer is not the output layer, then we use relu as the activation function.

Overfitting and Model Complexity

The two types of parameters used in machine learning are model parameters and hyperparameters. First, we will discuss model parameters. Model parameters are the parameters determined by the training set, the fitted parameters [8]. In our program, the parameters are trained using the images. The complexity of a model is proportional to the number of parameters: the larger number of parameters, the more complex the model is, and vice versa.

Hyperparameters are user chosen parameters that make the model better. By making the model better, we are talking about decreasing the loss and increasing the accuracy on the validation set. The hyperparameters that we considered in our program are the number of filters in the Conv2D layer, the kernel size in the Conv2D layer, the number of Conv2D layers, the number of Dense

layers, and the number of nodes in the Dense layer before the output layer (if there was one). The number of parameters varies from model to model depending on the hyperparameters chosen. Figure 2 displays a summary of the model we created that had the best results, which includes the number of parameters present.

Model: "sequential"		
Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 64, 64, 32)	320
max_pooling2d (MaxPooling2D)	(None, 32, 32, 32)	0
flatten (Flatten)	(None, 32768)	0
dense (Dense)	(None, 128)	4194432
dense_1 (Dense)	(None, 3)	387
Total params: 4,195,139		
Trainable params: 4,195,139		
Non-trainable params: 0		

Figure 2: Parameters in our final program

Hyperparameters also impact whether there is overfitting or not. Overfitting is when the error on the training set decreases but the error on the validation set increases. Ultimately, the goal is to have the error on the validation set as low as possible, in which overfitting does not help with the outcome of our program. A simple solution to overfitting is increasing the size of the training set, however that is not feasible when we were given a set amount of data to work with.

Results

In trying to create the model with the best results, we created various different Convolutional Neural Networks. Table 2 displays an example of one of the models that we created that did not produce the best results.

Layers	Results
1. Conv2D: 32 filters, kernel size 3, relu activation function , and same padding 2. MaxPool2D: pool size 2 3. Conv2D: 32 filters, kernel size 3, relu activation, and same padding 4. MaxPool2D: pool size 2 5. Flatten 6. Dense: 64 nodes and relu activation function 7. Dense: 3 nodes and softmax activation function	Accuracy on training set: 91.32% Accuracy on validation set: 71.58%

Table 2: One model that did not produce the best results

This is the layout of the model that produced the best results:

1. The first layer is convolutional with 32 filters. The kernel size is 3. The activation function is relu. The padding is the same.
2. The second layer is a pooling layer with a pool size of 2.
3. We then flatten the data and connect it to a dense layer with 128 nodes. The activation function is relu.
4. Finally, the output layer is a dense layer with 3 nodes and a softmax activation function.

We found the accuracy to be 96.45% on the training set and 78.42% on the validation set.

A baseline model for this program would be just to choose the more frequent category, so we will choose the Pneumonia category, which appears 280 times in the training set and 70 times in the validation set (after we combined the validation and test sets). Since there are 760 total examples in the training set, if we were to just choose Pneumonia as the category, we would be correct around 36.84% of the time. If we were to just choose Pneumonia in the validation set, which contains 190 examples, we would be correct 36.84% of the time.

Evaluating our final model in comparison to the baseline model, the final model had a much better output, practically doubling the accuracy the baseline had for the validation set.

Conclusions

Machine learning is a very useful field of computer science that has many applications in the world. In this paper, we demonstrated how to use machine learning to classify images. We also created many models and showcased the one with the best results in this paper.

If we were to repeat this type of program again, we would use a larger data set as that will help increase the accuracy in the model. A larger data set will also make the program more credible as there is a larger pool of data to verify the model.

Code

<https://colab.research.google.com/drive/1tYSOkmz1I6OZU76ub7AnlNeJsW5F-9Nm?usp=sharing>

References

1. https://colab.research.google.com/drive/19lPeK1TpGKSXT6ClXzNyySBSORC7r_5h?usp=sharing
2. Burns, Ed. "machine learning." *TechTarget*, www.techtarget.com. Accessed 26 Aug. 2022.
3. "CDC Museum COVID-19 Timeline." *Centers for Disease Control and Prevention*, www.cdc.gov. Accessed 31 Aug. 2022. Timeline.
4. Ganesh, Prakhar. "Types of Convolution Kernels : Simplified." *Towards Data Science*, 18 Oct. 2019, towardsdatascience.com. Accessed 26 Aug. 2022.
5. Kumar, Pranjall. (2022, August). Covid X-rays, Version 1. Retrieved August 31, 2022 from <https://www.kaggle.com/datasets/pranjallk1995/covid-xrays>
6. Mayo Clinic Staff. "Pneumonia." *Mayo Clinic*, 13 June 2020, www.mayoclinic.org. Accessed 31 Aug. 2022.
7. MRINALWALIA. "Keras.Conv2D Class." *GeeksforGeeks*, www.geeksforgeeks.org. Accessed 31 Aug. 2022.
8. Obi Tayo Ph.D, Benjamin. "Model Parameters and Hyperparameters in Machine Learning — What is the difference?" *Towards Data Science*, 30 Oct. 2019, towardsdatascience.com. Accessed 26 Aug. 2022.
9. Saha, Sumit. A Comprehensive Guide to Convolutional Neural Networks — the ELI5 way. *Towards Data Science*, 15 Dec. 2018, towardsdatascience.com. Accessed 26 Aug. 2022.
10. "tf.keras.layers.MaxPool2D." *TensorFlow*, www.tensorflow.org. Accessed 31 Aug. 2022.